KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

JAVA PROJECT REPORT ON

Hotel Room Reservation and Guest Management System

Submitted by

MOHIT KRISHNA K S

25BIT126

Under the Guidance of

MUGILAN R

Technical Trainer

Technical Department, KG College of Arts and Science

Academic Year: **2025 – 2026**

APRIL 2026

Semester: **II**

DECLARATION

I hereby declare that the project entitled "**Hotel Room Reservation and Guest Management System ”** submitted in partial fulfillment of the requirements for the course **JAVA PROGRAMMING** is a **record of my original work** carried out under the guidance of **MUGILAN R**, Technical Trainer, Technical Department, KG College of Arts and Science.

I further declare that this project has not been submitted previously, either in part or in full, for the award of any degree, diploma, or other similar title, at this or any other institution.

I have acknowledged all sources of information used in this project wherever applicable.

April 2026

Coimbatore

_____

(MOHIT KRISHNA K S)

CERTIFICATE

This is to certify that the project entitled **"Hotel Room Reservation and Guest Management System "** is a **bona fide work** carried out by Mohit Krishna K S **B.Sc-IT 'B' (2025-2028)**, **KG College of Arts & Science**, in partial fulfillment of the requirements for the course **Java Programming**.

This project has been completed under my supervision and guidance during the academic period **April 2026**.

Place: Coimbatore

Date:

Submitted for the Viva-Voce Examination held on _____

Guide:
(Technical Trainer, KGCAS)

Head of the Department
(Technical Head, KGCAS)

TABLE OF CONTENTS

S. No.	Topics	Page. No.
1	**Introduction**	
1.1	Objective	
1.2	Overview	
1.3	Features	
2	**Program Analysis**	
2.1	Problem Definition	
2.2	Project Overview	
2.3	Scopes and Limitations	
3	**System Design**	
3.1	Modules	
4	**Methodology**	
4.1	Problem Analysis	
4.2	System Design	
4.3	Program Development	
4.4	Testing and Validation	
4.5	Result and Documentation	
4.6	Algorithm	
4.7	Flowchart	
5	**Source Code**	
6	**Input and Output Screens**	
7	**Discussion and Conclusion**	
7.1	Discussion	
7.2	Conclusion	
8	**References**	

ABSTRACT

The **Hotel Reservation System** is a Java Swing application designed to automate the process of booking rooms, managing guest details, and handling reservations. Traditionally, many hotels rely on manual methods for maintaining records, which can lead to inefficiency, errors, and delays.

This project simplifies hotel operations by providing a digital platform where staff can manage reservations, check room availability, and generate booking confirmations. The system uses **Core Java concepts** such as classes, objects, methods, loops, conditional statements, and exception handling.

The main objective is to improve efficiency, reduce human errors, and provide faster service to customers. The project demonstrates the practical application of **object-oriented programming** in developing real-world business solutions.

1. INTRODUCTION

In the hotel industry, the efficiency of reservation and billing systems directly impacts customer satisfaction and operational success. Hotels and resorts often face challenges in managing room availability, guest records, and reservation confirmations, especially when these processes are handled manually. Traditional methods such as handwritten registers or basic spreadsheets are prone to errors, delays, and inconsistencies. These inefficiencies can lead to double-bookings, inaccurate billing, and poor customer experiences, ultimately affecting the reputation and profitability of the business.

With the rapid growth of technology, computerized reservation systems have become essential tools for modern hotels. They not only streamline operations but also ensure accuracy, speed, and reliability. By automating tasks such as room allocation, guest data entry, and invoice generation, hotels can reduce manual workload, minimize human errors, and provide faster service to customers.

The **Hotel Reservation System** developed in this project is a Java Swing application that demonstrates how programming concepts can be applied to solve real-world business problems. The system provides a user-friendly interface where hotel staff can manage reservations, check room availability, and generate booking confirmations with ease. It integrates fundamental **Core Java concepts** such as classes, objects, methods, loops, conditional statements, and exception handling, showcasing the practical application of object-oriented programming.

This project is designed not only as a functional solution but also as an educational exercise for understanding software development methodologies. It highlights the importance of structured program design, modular development, and systematic testing. The system is divided into clear modules — including room availability, guest management, reservation booking, and invoice generation — each responsible for a specific function. This modular approach makes the system easier to maintain, extend, and upgrade in the future.

Furthermore, the project emphasizes the role of graphical user interfaces (GUI) in enhancing usability. By using Java Swing, the system provides an interactive environment that is more intuitive than console-based applications. Staff can navigate menus, input guest details, and view booking confirmations in a visually organized manner, reducing training time and improving efficiency.

1.1 Objectives

The main objectives of the Hotel Reservation System are:

- To automate hotel room booking and reservation management.
- To reduce manual errors in guest records and billing.
- To provide a user-friendly GUI for hotel staff.
- To demonstrate Core Java concepts in a real-world application.

- To generate structured booking confirmations for customers.

1.2 Overview

The system is menu-driven with modules for guest management, room availability, and reservation handling. It generates structured booking confirmations and maintains records digitally. The application can be extended to include database integration and online booking features, making it scalable for larger hotel operations.

1.3 Features

The Hotel Reservation System includes the following features:

- **Room Availability Check** – Displays available rooms and prevents double-booking.
- **Guest Record Management** – Stores guest details for easy retrieval.
- **Reservation Booking and Cancellation** – Allows staff to add or cancel reservations quickly.
- **Invoice/Confirmation Generation** – Produces a detailed booking confirmation for customers.
- **User-Friendly Interface** – Built with Java Swing for intuitive navigation.
- **Fast Processing** – Reduces the time required to manage reservations.

The **primary goal** of this project is to demonstrate how technology can transform traditional hotel operations into streamlined, error-free processes. While the current version focuses on core reservation and billing functionalities, it lays the foundation for future enhancements such as database integration, online booking portals, and payment gateways.

2. PROGRAM ANALYSIS

Program analysis is a critical phase in the software development lifecycle. It involves studying the problem in detail, identifying system requirements, and defining the scope of the solution before moving into design and implementation. Without proper analysis, projects often suffer from unclear objectives, incomplete features, or inefficient workflows.

For the **Hotel Reservation System**, program analysis helps in understanding the challenges faced by hotels in managing reservations manually and how a computerized solution can address these issues. By analyzing the problem, the project ensures that the system is designed to meet the real needs of hotel staff and customers. This phase also clarifies the functionalities required, such as room availability checks, guest record management, reservation booking, and invoice generation.

Through program analysis, the project establishes a clear roadmap:

- Define the problems in existing manual systems.
- Provide an overview of the proposed solution.
- Identify the scope of the system and acknowledge its limitations.

This structured approach ensures that the Hotel Reservation System is not only technically sound but also practically useful in real-world hospitality environments.

2.1 Problem Definition

In many hotels and guest houses, reservations are still managed manually using registers or spreadsheets. This traditional approach creates several challenges:

- **Errors in booking** due to human miscalculation or oversight.
- **Double-bookings** when multiple staff members handle reservations without a centralized system.
- **Difficulty in managing guest records**, especially during peak seasons.
- **Lack of structured invoices or confirmations**, leading to confusion for both staff and customers.
- **Time-consuming processes** that reduce efficiency and customer satisfaction.

These issues highlight the need for an automated system that can streamline reservation management, ensure accuracy, and provide faster service.

2.2 Project Overview

The **Hotel Reservation System** is a Java Swing application designed to automate hotel booking operations. The system provides a graphical interface where staff can:

- Check room availability in real time.
- Add new reservations with guest details.
- Cancel or update existing reservations.
- Generate structured booking confirmations or invoices.

The project demonstrates the use of **Core Java concepts** such as classes, objects, methods, loops, conditional statements, and exception handling. By organizing the application into modules (Room Availability, Guest Management, Reservation, Invoice Generation, Exit), the system ensures clarity, maintainability, and scalability.

This project not only solves real-world operational problems but also serves as a practical demonstration of object-oriented programming principles applied to business applications.

2.3 Scopes and Limitations

Scope The Hotel Reservation System can be used in:

- Hotels and resorts
- Guest houses and lodges
- Hostels and service apartments
- Small hospitality businesses

Its functionalities include:

- Managing room availability
- Handling guest records
- Automating reservation booking and cancellation
- Generating booking confirmations

Limitations

- Currently designed for single-hotel use; does not support multi-branch operations.
- No database integration — records are session-based and not stored persistently.
- No online booking or payment gateway support.
- Limited to desktop usage; not yet available as a web or mobile application.

3. SYSTEM DESIGN

System design is the stage where the overall structure of the application is planned before coding begins. It defines how different components of the system interact, how data flows between modules, and how the user experiences the application. A well-designed system ensures efficiency, scalability, and ease of maintenance.

For the **Hotel Reservation System**, the design focuses on creating a modular, menu-driven application that allows hotel staff to manage reservations seamlessly. Each module is responsible for a specific function, such as displaying room availability, handling guest records, processing reservations, and generating invoices. By dividing the system into modules, the project achieves clarity in functionality, reduces complexity, and makes future enhancements easier to implement.

The design also emphasizes the importance of a graphical user interface (GUI) using Java Swing. Unlike console-based applications, a GUI provides an intuitive environment where staff can interact with the system through menus, buttons, and forms. This improves usability and reduces training requirements.

3.1 Modules

The Hotel Reservation System is divided into the following modules:

1. **Room Availability Module**
    - Displays available rooms in the hotel.
    - Prevents double-booking by updating room status once reserved.
    - Provides real-time availability information to staff.
2. **Guest Management Module**
    - Records guest details such as name, contact information, and stay duration.
    - Allows staff to update or retrieve guest records easily.
    - Ensures organized storage of guest information for future reference.
3. **Reservation Module**
    - Handles booking and cancellation of reservations.
    - Links guest details with room availability.
    - Provides confirmation messages once a reservation is completed.
4. **Invoice/Confirmation Module**
    - Generates a structured booking confirmation or invoice.
    - Displays details such as guest name, room type, duration of stay, and total charges.

- Ensures transparency and accuracy in billing.

5. **Exit Module**
    - Allows staff to safely close the application after completing tasks.
    - Ensures that all operations are finalized before termination.

4. METHODOLOGY

Methodology describes the structured process followed to develop the project. It ensures that the system is built systematically, from understanding the problem to designing, coding, testing, and documenting the results. For the **Hotel Reservation System**, the methodology highlights how each stage contributes to building a reliable and efficient application.

4.1 Problem Analysis

In this phase, the existing manual reservation process was studied. The main problems identified include:

- Manual calculations leading to booking errors.
- Double-bookings due to lack of centralized records.
- Difficulty in managing guest details efficiently.
- Time-consuming reservation handling during peak hours.

Based on these issues, the requirements for an automated reservation system were defined: accuracy, speed, user-friendliness, and structured record management.

4.2 System Design

The system was divided into modules to simplify development and maintenance:

- **Room Availability Module** – Displays available rooms and prevents double-booking.
- **Guest Management Module** – Records and retrieves guest details.
- **Reservation Module** – Handles booking and cancellation.
- **Invoice/Confirmation Module** – Generates structured booking confirmations.
- **Exit Module** – Safely terminates the application.

This modular design ensures clarity and scalability.

4.3 Program Development

The program was implemented using **Java Swing** for GUI and Core Java concepts for logic. Key steps included:

- Creating classes and methods for each module.
- Implementing menu-driven logic for navigation.
- Writing code for reservation handling and invoice generation.
- Handling user input with validation and exception handling.

4.4 Testing and Validation

Testing was conducted to ensure accuracy and reliability. Example test cases:

Test Case	Expected Result
Select room	Room displayed correctly
Enter guest details	Correct details accepted
Confirm reservation	Accurate booking confirmation generated
Invalid input	Error handled gracefully

Validation confirmed that the system performs as expected under different scenarios.

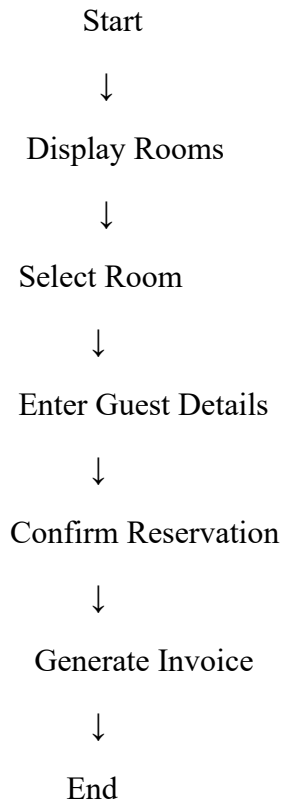
4.5 Result and Documentation

The system successfully generates accurate booking confirmations and manages reservations efficiently. Documentation explains the design, implementation, and usage of the system, supported by screenshots and code samples.

4.6 Algorithm

1. Start the program
2. Display available rooms
3. Select room
4. Enter guest details
5. Confirm reservation
6. Generate invoice/confirmation
7. End program

4.7 Flowchart



5. SOURCE CODE

```
package com.mycompany.Backend;


import java.io.*;
import java.util.*;


public class FileHandler {


   public static void writeLine(String filename, String data) {
      try (BufferedWriter bw = new BufferedWriter(new FileWriter(filename, true))) {
         bw.write(data);
         bw.newLine();
      } catch (IOException e) {
         System.err.println("Error writing to file '" + filename + "': " + e.getMessage());
      }
   }


   public static List<String> readAll(String filename) {
      List<String> lines = new ArrayList<>();
      try (BufferedReader br = new BufferedReader(new FileReader(filename))) {
         String line;
         while ((line = br.readLine()) != null) {
            lines.add(line);
         }
      } catch (FileNotFoundException e) {
         System.err.println("File not found: " + filename + ". A new file will be created when
writing.");
      } catch (IOException e) {
```

```
            System.err.println("Error reading file '" + filename + "': " + e.getMessage());
        }
        return lines;
    }

    public static void overwrite(String filename, List<String> lines) {
        try (BufferedWriter bw = new BufferedWriter(new FileWriter(filename))) {
            for (String line : lines) {
                bw.write(line);
                bw.newLine();
            }
        } catch (IOException e) {
            System.err.println("Error overwriting file '" + filename + "': " + e.getMessage());
        }
    }
}
```

```
package com.mycompany.Backend;
public class Guest {
    private int id;
    private String name;
    private String phone;
    private String email;

    // Constructor
    public Guest(int id, String name, String phone, String email) {
        this.id = id;
        this.name = name;
```

```
    this.phone = phone;
    this.email = email;
}

// Getters
public int getId() {
    return id;
}

public String getName() {
    return name;
}

public String getPhone() {
    return phone;
}

public String getEmail() {
    return email;
}

// Setters
public void setId(int id) {
    this.id = id;
}

public void setName(String name) {
    this.name = name;
}
```

```
    public void setPhone(String phone) {
        this.phone = phone;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    // toString for easy printing
    @Override
    public String toString() {
        return "Guest [ID=" + id + ", Name=" + name + ", Phone=" + phone + ", Email=" +
email + "]";
    }
}
```

```
package com.mycompany.Backend;
import java.util.*;
import java.io.FileReader;
import java.io.IOException;
import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.FileWriter;

public class GuestService {
private static final String FILE = "guests.txt";

public void addGuest(Guest g) {
String record = g.getId() + "|" + g.getName() + "|" + g.getPhone() + "|" + g.getEmail();
try (BufferedWriter bw = new BufferedWriter(new FileWriter("guests.txt", true))) {
bw.write(record);
bw.newLine(); // ensures each guest is on its own line
```

```
System.out.println("Record written: " + record);
} catch (IOException e) {
e.printStackTrace();
}
}


public List<Guest> getAllGuests() {
List<Guest> guests = new ArrayList<>();
try (BufferedReader br = new BufferedReader(new FileReader("guests.txt"))) {
String line;
while ((line = br.readLine()) != null) {
String[] parts = line.split("\\|");
if (parts.length == 4) {
guests.add(new Guest(
Integer.parseInt(parts[0]),
parts[1],
parts[2],
parts[3]
));
}
}
} catch (IOException e) {
e.printStackTrace();
}
return guests;
}


public void updateGuest(int id, Guest updated) {
List<String> lines = FileHandler.readAll(FILE);
List<String> newLines = new ArrayList<>();
boolean found = false;

for (String line : lines) {
String[] parts = line.split("\\|");
try {
int currentId = Integer.parseInt(parts[0]);
if (currentId == id) {
newLines.add(updated.getId() + "|" + updated.getName() + "|" + updated.getPhone() + "|" +
updated.getEmail());
found = true;
} else {
```

```
newLines.add(line);
}
} catch (NumberFormatException e) {
System.err.println("Invalid ID format in record: " + line);
newLines.add(line); // keep original line
}
}

if (!found) {
System.err.println("Guest with ID " + id + " not found.");
}

FileHandler.overwrite(FILE, newLines);
}

public void deleteGuest(int id) {
List<String> lines = FileHandler.readAll(FILE);
List<String> newLines = new ArrayList<>();
boolean deleted = false;

for (String line : lines) {
String[] parts = line.split("\\|");
try {
int currentId = Integer.parseInt(parts[0]);
if (currentId != id) {
newLines.add(line);
} else {
deleted = true;
}
} catch (NumberFormatException e) {
System.err.println("Invalid ID format in record: " + line);
newLines.add(line); // keep original line
}
}

if (!deleted) {
System.err.println("Guest with ID " + id + " not found.");
}

FileHandler.overwrite(FILE, newLines);
}
}
```

```
package com.mycompany.Backend;
import java.time.LocalDate;

public class Payment {
    private int id;
    private int reservationId;   // Link to Reservation
    private double amount;
    private String method;       // e.g., "Cash", "Card", "UPI"
    private LocalDate date;

    // Constructor
    public Payment(int id, int reservationId, double amount, String method, LocalDate date) {
        this.id = id;
        this.reservationId = reservationId;
        this.amount = amount;
        this.method = method;
        this.date = date;
    }

    // Getters & Setters
    public int getId() { return id; }
    public int getReservationId() { return reservationId; }
    public double getAmount() { return amount; }
    public String getMethod() { return method; }
    public LocalDate getDate() { return date; }

    public void setId(int id) { this.id = id; }
    public void setReservationId(int reservationId) { this.reservationId = reservationId; }
    public void setAmount(double amount) { this.amount = amount; }
    public void setMethod(String method) { this.method = method; }
    public void setDate(LocalDate date) { this.date = date; }

    @Override
    public String toString() {
        return "Payment [ID=" + id + ", ReservationID=" + reservationId +
                ", Amount=" + amount + ", Method=" + method + ", Date=" + date + "]";
    }
```

```
}
```

```
package com.mycompany.Backend;

import java.time.LocalDate;
import java.util.*;

public class PaymentService {
    private static final String FILE = "payments.txt";

    // Add a payment
    public void addPayment(Payment p) {
        String record = p.getId() + "|" + p.getReservationId() + "|" +
                        p.getAmount() + "|" + p.getMethod() + "|" + p.getDate();
        FileHandler.writeLine(FILE, record);
    }

    // Get all payments
    public List<Payment> getAllPayments() {
        List<Payment> payments = new ArrayList<>();
        List<String> lines = FileHandler.readAll(FILE);

        for (String line : lines) {
            String[] parts = line.split("\\|");
            if (parts.length == 5) {
                try {
                    int id = Integer.parseInt(parts[0]);
                    int reservationId = Integer.parseInt(parts[1]);
                    double amount = Double.parseDouble(parts[2]);
                    String method = parts[3];
                    LocalDate date = LocalDate.parse(parts[4]);
                    payments.add(new Payment(id, reservationId, amount, method, date));
                } catch (Exception e) {
                    System.err.println("Invalid payment record: " + line);
                }
            } else {
                System.err.println("Skipping invalid payment record: " + line);
            }
        }
```

```
        return payments;
    }

    // Delete a payment
    public void deletePayment(int id) {
        List<String> lines = FileHandler.readAll(FILE);
        List<String> newLines = new ArrayList<>();
        boolean deleted = false;

        for (String line : lines) {
            String[] parts = line.split("\\|");
            try {
                int currentId = Integer.parseInt(parts[0]);
                if (currentId != id) {
                    newLines.add(line);
                } else {
                    deleted = true;
                }
            } catch (NumberFormatException e) {
                System.err.println("Invalid ID format in payment record: " + line);
                newLines.add(line);
            }
        }

        if (!deleted) {
            System.err.println("Payment with ID " + id + " not found.");
        }

        FileHandler.overwrite(FILE, newLines);
    }
}
```

```
package com.mycompany.Backend;

import java.time.LocalDate;

public class Reservation {
    private String reservationId;   // unique identifier
    private String guestId;         // guest identifier
    private String roomNumber;      // room number (String)
```

```
    private LocalDate checkIn;
    private LocalDate checkOut;

    public Reservation(String reservationId, String guestId, String roomNumber,
                       LocalDate checkIn, LocalDate checkOut) {
        this.reservationId = reservationId;
        this.guestId = guestId;
        this.roomNumber = roomNumber;
        this.checkIn = checkIn;
        this.checkOut = checkOut;
    }

    public String getReservationId() { return reservationId; }
    public String getGuestId() { return guestId; }
    public String getRoomNumber() { return roomNumber; }
    public LocalDate getCheckIn() { return checkIn; }
    public LocalDate getCheckOut() { return checkOut; }

    @Override
    public String toString() {
        return reservationId + "|" + guestId + "|" + roomNumber + "|" + checkIn + "|" +
checkOut;
    }
}
```

```
package com.mycompany.Backend;

import java.io.*;
import java.time.LocalDate;
import java.util.*;

public class ReservationService {
private static final String FILE = "reservations.txt";
private final RoomService roomService = new RoomService();

// Add reservation
public void addReservation(Reservation r) {
Room room = roomService.getRoomByNumber(r.getRoomNumber());
if (room == null) {
```

```
System.err.println("Room not found: " + r.getRoomNumber());
return;
}
if (!room.getStatus().equals("Available")) {
System.err.println("Room not available: " + r.getRoomNumber());
return;
}

try (PrintWriter pw = new PrintWriter(new FileWriter(FILE, true))) {
pw.println(r.toString());
} catch (IOException e) {
e.printStackTrace();
}

roomService.updateRoomStatus(r.getRoomNumber(), "Booked");
}

// Get all reservations
public List<Reservation> getAllReservations() {
List<Reservation> reservations = new ArrayList<>();
try (BufferedReader br = new BufferedReader(new FileReader(FILE))) {
String line;
while ((line = br.readLine()) != null) {
String[] parts = line.split("\\|");
if (parts.length == 5) {
String reservationId = parts[0];
String guestId = parts[1];
String roomNumber = parts[2];
LocalDate checkIn = LocalDate.parse(parts[3]);
LocalDate checkOut = LocalDate.parse(parts[4]);

reservations.add(new Reservation(reservationId, guestId, roomNumber, checkIn, checkOut));
}
}
} catch (IOException e) {
e.printStackTrace();
}
return reservations;
}

// Delete reservation by ID
public void deleteReservation(String reservationId) {
List<Reservation> reservations = getAllReservations();
```

```
Reservation target = null;

try (PrintWriter pw = new PrintWriter(new FileWriter(FILE))) {
for (Reservation r : reservations) {
if (!r.getReservationId().equals(reservationId)) {
pw.println(r.toString());
} else {
target = r;
}
}
} catch (IOException e) {
e.printStackTrace();
}

if (target != null) {
roomService.updateRoomStatus(target.getRoomNumber(), "Available");
}
}
}
```

```
package com.mycompany.Backend;

public class Room {
    private String number;   // e.g., "101", "A-203"
    private String type;     // e.g., "Single", "Double", "Suite"
    private double price;    // nightly rate
    private String status;   // "Available", "Booked", "Maintenance"

    public Room(String number, String type, double price, String status) {
        this.number = number;
        this.type = type;
        this.price = price;
        this.status = status;
    }

    public String getNumber() { return number; }
    public String getType() { return type; }
    public double getPrice() { return price; }
    public String getStatus() { return status; }

    public void setNumber(String number) { this.number = number; }
```

```
    public void setType(String type) { this.type = type; }
    public void setPrice(double price) { this.price = price; }
    public void setStatus(String status) { this.status = status; }

    @Override
    public String toString() {
        return "Room [Number=" + number + ", Type=" + type +
                ", Price=" + price + ", Status=" + status + "]";
    }
}
```

```
package com.mycompany.Backend;

import java.io.*;
import java.util.*;

public class RoomService {
private static final String FILE = "rooms.txt";

// Add a new room
public void addRoom(Room r) {
try (PrintWriter pw = new PrintWriter(new FileWriter(FILE, true))) {
pw.println(r.getNumber() + "|" + r.getType() + "|" + r.getPrice() + "|" + r.getStatus());
} catch (IOException e) {
e.printStackTrace();
}
}

// Get all rooms
public List<Room> getAllRooms() {
List<Room> rooms = new ArrayList<>();
try (BufferedReader br = new BufferedReader(new FileReader(FILE))) {
String line;
while ((line = br.readLine()) != null) {
String[] parts = line.split("\\|");
if (parts.length == 4) {
try {
String number = parts[0];
String type = parts[1];
double price = Double.parseDouble(parts[2]);
```

```
String status = parts[3];
rooms.add(new Room(number, type, price, status));
} catch (NumberFormatException e) {
System.err.println("Invalid price format in record: " + line);
}
} else {
System.err.println("Skipping invalid room record: " + line);
}
}
} catch (IOException e) {
e.printStackTrace();
}
return rooms;
}

// Delete a room by number
public void deleteRoom(String number) {
List<Room> rooms = getAllRooms();
try (PrintWriter pw = new PrintWriter(new FileWriter(FILE))) {
for (Room r : rooms) {
if (!r.getNumber().equals(number)) {
pw.println(r.getNumber() + "|" + r.getType() + "|" + r.getPrice() + "|" + r.getStatus());
}
}
} catch (IOException e) {
e.printStackTrace();
}
}

// Update room status (availability)
public void updateRoomStatus(String number, String newStatus) {
List<Room> rooms = getAllRooms();
try (PrintWriter pw = new PrintWriter(new FileWriter(FILE))) {
for (Room r : rooms) {
if (r.getNumber().equals(number)) {
r.setStatus(newStatus);
}
pw.println(r.getNumber() + "|" + r.getType() + "|" + r.getPrice() + "|" + r.getStatus());
}
} catch (IOException e) {
e.printStackTrace();
}
}
```

```
// Find a room by number
public Room getRoomByNumber(String number) {
for (Room r : getAllRooms()) {
if (r.getNumber().equals(number)) {
return r;
}
}
return null;
}
}
```

GUI\

```
package com.mycompany.GUI;


import com.mycompany.Backend.Guest;
import com.mycompany.Backend.GuestService;
import javax.swing.JOptionPane;
import javax.swing.table.DefaultTableModel;

/**
*
* @author KGCAS
*/
public class GuestFrame extends javax.swing.JFrame {
private GuestService guestService = new GuestService();
/**
* Creates new form GuestFrame
*/
public GuestFrame() {
initComponents();
txtGuestId.addActionListener(e -> txtGuestName.requestFocus());
txtGuestName.addActionListener(e -> txtGuestPhone.requestFocus());
txtGuestPhone.addActionListener(e -> txtGuestEmail.requestFocus());
txtGuestEmail.addActionListener(e -> BtnAddGuest.doClick());

}
```

```
/**
* This method is called from within the constructor to initialize the form.
* WARNING: Do NOT modify this code. The content of this method is always
* regenerated by the Form Editor.
*/
@SuppressWarnings("unchecked")
// <editor-fold defaultstate="collapsed" desc="Generated Code">//GEN-
BEGIN:initComponents
private void initComponents() {

jScrollPane1 = new javax.swing.JScrollPane();
jTable1 = new javax.swing.JTable();
jLabel1 = new javax.swing.JLabel();
jLabel2 = new javax.swing.JLabel();
jLabel3 = new javax.swing.JLabel();
jLabel4 = new javax.swing.JLabel();
jLabel5 = new javax.swing.JLabel();
txtGuestId = new javax.swing.JTextField();
txtGuestName = new javax.swing.JTextField();
txtGuestPhone = new javax.swing.JTextField();
txtGuestEmail = new javax.swing.JTextField();
jScrollPane2 = new javax.swing.JScrollPane();
tblGuests = new javax.swing.JTable();
BtnAddGuest = new javax.swing.JButton();
BtnViewGuests = new javax.swing.JButton();
BtnBack = new javax.swing.JButton();

jTable1.setModel(new javax.swing.table.DefaultTableModel(
new Object [][] {
{null, null, null, null},
{null, null, null, null},
{null, null, null, null},
{null, null, null, null}
},
new String [] {
"Title 1", "Title 2", "Title 3", "Title 4"
}
));
jScrollPane1.setViewportView(jTable1);

setDefaultCloseOperation(javax.swing.WindowConstants.EXIT_ON_CLOSE);
```

```
jLabel1.setFont(new java.awt.Font("Segoe UI", 0, 24)); // NOI18N
jLabel1.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
jLabel1.setText("Guest Management");

jLabel2.setFont(new java.awt.Font("Segoe UI", 1, 12)); // NOI18N
jLabel2.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);
jLabel2.setText("Guest ID");

jLabel3.setFont(new java.awt.Font("Segoe UI", 1, 12)); // NOI18N
jLabel3.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);
jLabel3.setText("Guest Name");

jLabel4.setFont(new java.awt.Font("Segoe UI", 1, 12)); // NOI18N
jLabel4.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);
jLabel4.setText("Phone No.");

jLabel5.setFont(new java.awt.Font("Segoe UI", 1, 12)); // NOI18N
jLabel5.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);
jLabel5.setText("Email");

txtGuestId.addActionListener(new java.awt.event.ActionListener() {
public void actionPerformed(java.awt.event.ActionEvent evt) {
txtGuestIdActionPerformed(evt);
}
});

txtGuestName.addActionListener(new java.awt.event.ActionListener() {
public void actionPerformed(java.awt.event.ActionEvent evt) {
txtGuestNameActionPerformed(evt);
}
});

txtGuestPhone.addActionListener(new java.awt.event.ActionListener() {
public void actionPerformed(java.awt.event.ActionEvent evt) {
txtGuestPhoneActionPerformed(evt);
}
});

txtGuestEmail.addActionListener(new java.awt.event.ActionListener() {
public void actionPerformed(java.awt.event.ActionEvent evt) {
txtGuestEmailActionPerformed(evt);
}
});
```

```
tblGuests.setModel(new javax.swing.table.DefaultTableModel(
new Object [][] {
{null, null, null, null},
{null, null, null, null},
{null, null, null, null},
{null, null, null, null}
},
new String [] {
"Guest ID", "Guest Name", "Phone No.", "Email"
}
));
jScrollPane2.setViewportView(tblGuests);

BtnAddGuest.setText("Add Guest");
BtnAddGuest.addActionListener(new java.awt.event.ActionListener() {
public void actionPerformed(java.awt.event.ActionEvent evt) {
BtnAddGuestActionPerformed(evt);
}
});

BtnViewGuests.setText("Show Guests");
BtnViewGuests.addActionListener(new java.awt.event.ActionListener() {
public void actionPerformed(java.awt.event.ActionEvent evt) {
BtnViewGuestsActionPerformed(evt);
}
});

BtnBack.setText("Exit");
BtnBack.addActionListener(new java.awt.event.ActionListener() {
public void actionPerformed(java.awt.event.ActionEvent evt) {
BtnBackActionPerformed(evt);
}
});

javax.swing.GroupLayout layout = new javax.swing.GroupLayout(getContentPane());
getContentPane().setLayout(layout);
layout.setHorizontalGroup(
layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
.addGroup(layout.createSequentialGroup()
.addContainerGap()
.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
.addComponent(jScrollPane2, javax.swing.GroupLayout.Alignment.TRAILING,
```

```
javax.swing.GroupLayout.DEFAULT_SIZE, 483, Short.MAX_VALUE)
.addComponent(jLabel1, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
.addGroup(layout.createSequentialGroup()
.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.TRAILING,
false)
.addComponent(jLabel4, javax.swing.GroupLayout.Alignment.LEADING,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE,
Short.MAX_VALUE)
.addComponent(jLabel3, javax.swing.GroupLayout.Alignment.LEADING,
javax.swing.GroupLayout.DEFAULT_SIZE, 81, Short.MAX_VALUE)
.addComponent(jLabel2, javax.swing.GroupLayout.Alignment.LEADING,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE,
Short.MAX_VALUE)
.addComponent(jLabel5, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE))
.addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)
.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING,
false)
.addComponent(txtGuestId, javax.swing.GroupLayout.DEFAULT_SIZE, 301,
Short.MAX_VALUE)
.addComponent(txtGuestName)
.addComponent(txtGuestPhone)
.addComponent(txtGuestEmail))
.addGap(0, 0, Short.MAX_VALUE))
.addGroup(layout.createSequentialGroup()
.addComponent(BtnAddGuest, javax.swing.GroupLayout.PREFERRED_SIZE, 135,
javax.swing.GroupLayout.PREFERRED_SIZE)
.addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
.addComponent(BtnViewGuests, javax.swing.GroupLayout.PREFERRED_SIZE, 135,
javax.swing.GroupLayout.PREFERRED_SIZE)
.addGap(36, 36, 36)
.addComponent(BtnBack, javax.swing.GroupLayout.PREFERRED_SIZE, 135,
javax.swing.GroupLayout.PREFERRED_SIZE)))
.addContainerGap())
);
layout.setVerticalGroup(
layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
.addGroup(layout.createSequentialGroup()
.addContainerGap()
.addComponent(jLabel1, javax.swing.GroupLayout.PREFERRED_SIZE, 39,
javax.swing.GroupLayout.PREFERRED_SIZE)
```

```
.addGap(21, 21, 21)
.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
.addComponent(jLabel2)
.addComponent(txtGuestId, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.PREFERRED_SIZE))
.addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.UNRELATED)
.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
.addComponent(txtGuestName, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE)
.addComponent(jLabel3))
.addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.UNRELATED)
.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
.addComponent(jLabel4)
.addComponent(txtGuestPhone, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.PREFERRED_SIZE))
.addGap(15, 15, 15)
.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
.addComponent(jLabel5, javax.swing.GroupLayout.PREFERRED_SIZE, 22,
javax.swing.GroupLayout.PREFERRED_SIZE)
.addComponent(txtGuestEmail, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.PREFERRED_SIZE))
.addGap(18, 18, 18)
.addComponent(jScrollPane2, javax.swing.GroupLayout.PREFERRED_SIZE, 158,
javax.swing.GroupLayout.PREFERRED_SIZE)
.addGap(18, 18, 18)
.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
.addComponent(BtnAddGuest, javax.swing.GroupLayout.PREFERRED_SIZE, 30,
javax.swing.GroupLayout.PREFERRED_SIZE)
.addComponent(BtnViewGuests, javax.swing.GroupLayout.PREFERRED_SIZE, 30,
javax.swing.GroupLayout.PREFERRED_SIZE)
.addComponent(BtnBack, javax.swing.GroupLayout.PREFERRED_SIZE, 30,
javax.swing.GroupLayout.PREFERRED_SIZE))
.addContainerGap(19, Short.MAX_VALUE))
);

pack();
}// </editor-fold>//GEN-END:initComponents

private void txtGuestNameActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
FIRST:event_txtGuestNameActionPerformed
```

```
// TODO add your handling code here:
}//GEN-LAST:event_txtGuestNameActionPerformed

private void txtGuestPhoneActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
FIRST:event_txtGuestPhoneActionPerformed
// TODO add your handling code here:
}//GEN-LAST:event_txtGuestPhoneActionPerformed

private void txtGuestEmailActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
FIRST:event_txtGuestEmailActionPerformed
// TODO add your handling code here:
}//GEN-LAST:event_txtGuestEmailActionPerformed

private void txtGuestIdActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
FIRST:event_txtGuestIdActionPerformed
// TODO add your handling code here:
}//GEN-LAST:event_txtGuestIdActionPerformed

private void BtnAddGuestActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
FIRST:event_BtnAddGuestActionPerformed
Guest g = new Guest(
Integer.parseInt(txtGuestId.getText()),
txtGuestName.getText(),
txtGuestPhone.getText(),
txtGuestEmail.getText()
);
guestService.addGuest(g);
JOptionPane.showMessageDialog(this, "Guest added successfully!", "Success",
JOptionPane.INFORMATION_MESSAGE);
}//GEN-LAST:event_BtnAddGuestActionPerformed

private void BtnViewGuestsActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
FIRST:event_BtnViewGuestsActionPerformed
DefaultTableModel model = (DefaultTableModel) tblGuests.getModel();
model.setRowCount(0);
for (Guest g : guestService.getAllGuests()) {
model.addRow(new Object[]{
g.getId(),
g.getName(),
g.getPhone(),
g.getEmail()
});
}
```

```
}//GEN-LAST:event_BtnViewGuestsActionPerformed

private void BtnBackActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
FIRST:event_BtnBackActionPerformed
dispose();
}//GEN-LAST:event_BtnBackActionPerformed

/**
* @param args the command line arguments
*/
public static void main(String args[]) {
/* Set the Nimbus look and feel */
//<editor-fold defaultstate="collapsed" desc=" Look and feel setting code (optional) ">
/* If Nimbus (introduced in Java SE 6) is not available, stay with the default look and feel.
* For details see http://download.oracle.com/javase/tutorial/uiswing/lookandfeel/plaf.html
*/
try {
for (javax.swing.UIManager.LookAndFeelInfo info :
javax.swing.UIManager.getInstalledLookAndFeels()) {
if ("Nimbus".equals(info.getName())) {
javax.swing.UIManager.setLookAndFeel(info.getClassName());
break;
}
}
} catch (ClassNotFoundException ex) {
java.util.logging.Logger.getLogger(GuestFrame.class.getName()).log(java.util.logging.Level.
SEVERE, null, ex);
} catch (InstantiationException ex) {
java.util.logging.Logger.getLogger(GuestFrame.class.getName()).log(java.util.logging.Level.
SEVERE, null, ex);
} catch (IllegalAccessException ex) {
java.util.logging.Logger.getLogger(GuestFrame.class.getName()).log(java.util.logging.Level.
SEVERE, null, ex);
} catch (javax.swing.UnsupportedLookAndFeelException ex) {
java.util.logging.Logger.getLogger(GuestFrame.class.getName()).log(java.util.logging.Level.
SEVERE, null, ex);
}
//</editor-fold>

/* Create and display the form */
java.awt.EventQueue.invokeLater(new Runnable() {
public void run() {
new GuestFrame().setVisible(true);
```

```
}
});
}

// Variables declaration - do not modify//GEN-BEGIN:variables
private javax.swing.JButton BtnAddGuest;
private javax.swing.JButton BtnBack;
private javax.swing.JButton BtnViewGuests;
private javax.swing.JLabel jLabel1;
private javax.swing.JLabel jLabel2;
private javax.swing.JLabel jLabel3;
private javax.swing.JLabel jLabel4;
private javax.swing.JLabel jLabel5;
private javax.swing.JScrollPane jScrollPane1;
private javax.swing.JScrollPane jScrollPane2;
private javax.swing.JTable jTable1;
private javax.swing.JTable tblGuests;
private javax.swing.JTextField txtGuestEmail;
private javax.swing.JTextField txtGuestId;
private javax.swing.JTextField txtGuestName;
private javax.swing.JTextField txtGuestPhone;
// End of variables declaration//GEN-END:variables
}
```

```
/*
* Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
* Click nbfs://nbhost/SystemFileSystem/Templates/GUIForms/JFrame.java to edit this template
*/
package com.mycompany.GUI;


/**
*
* @author KGCAS
*/
public class MainMenu extends javax.swing.JFrame {

/**
```

```
* Creates new form MainMenu
*/
public MainMenu() {
initComponents();
}

/**
* This method is called from within the constructor to initialize the form.
* WARNING: Do NOT modify this code. The content of this method is always
* regenerated by the Form Editor.
*/
@SuppressWarnings("unchecked")
// <editor-fold defaultstate="collapsed" desc="Generated Code">//GEN-
BEGIN:initComponents
private void initComponents() {

BtnGuest = new javax.swing.JButton();
BtnReservation = new javax.swing.JButton();
BtnRoom = new javax.swing.JButton();
BtnPayment = new javax.swing.JButton();
BtnExit = new javax.swing.JButton();
jLabel1 = new javax.swing.JLabel();

setDefaultCloseOperation(javax.swing.WindowConstants.EXIT_ON_CLOSE);

BtnGuest.setText("Guest Service");
BtnGuest.addActionListener(new java.awt.event.ActionListener() {
public void actionPerformed(java.awt.event.ActionEvent evt) {
BtnGuestActionPerformed(evt);
}
});

BtnReservation.setText("Reservation Service");
BtnReservation.addActionListener(new java.awt.event.ActionListener() {
public void actionPerformed(java.awt.event.ActionEvent evt) {
BtnReservationActionPerformed(evt);
}
});

BtnRoom.setText("Room Service");
BtnRoom.addActionListener(new java.awt.event.ActionListener() {
public void actionPerformed(java.awt.event.ActionEvent evt) {
BtnRoomActionPerformed(evt);
```

```
}
});

BtnPayment.setText("Payment Service");
BtnPayment.addActionListener(new java.awt.event.ActionListener() {
public void actionPerformed(java.awt.event.ActionEvent evt) {
BtnPaymentActionPerformed(evt);
}
});

BtnExit.setText("Exit");
BtnExit.addActionListener(new java.awt.event.ActionListener() {
public void actionPerformed(java.awt.event.ActionEvent evt) {
BtnExitActionPerformed(evt);
}
});

jLabel1.setFont(new java.awt.Font("Segoe UI", 1, 18)); // NOI18N
jLabel1.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
jLabel1.setText("Reservation and Management");

javax.swing.GroupLayout layout = new javax.swing.GroupLayout(getContentPane());
getContentPane().setLayout(layout);
layout.setHorizontalGroup(
layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
.addGroup(layout.createSequentialGroup()
.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
.addGroup(layout.createSequentialGroup()
.addContainerGap()
.addComponent(jLabel1, javax.swing.GroupLayout.DEFAULT_SIZE, 772,
Short.MAX_VALUE))
.addGroup(layout.createSequentialGroup()
.addGap(321, 321, 321)
.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.TRAILING)
.addComponent(BtnReservation)
.addComponent(BtnRoom, javax.swing.GroupLayout.PREFERRED_SIZE, 132,
javax.swing.GroupLayout.PREFERRED_SIZE)
.addComponent(BtnPayment, javax.swing.GroupLayout.PREFERRED_SIZE, 132,
javax.swing.GroupLayout.PREFERRED_SIZE)
.addComponent(BtnExit, javax.swing.GroupLayout.PREFERRED_SIZE, 132,
javax.swing.GroupLayout.PREFERRED_SIZE)
.addComponent(BtnGuest, javax.swing.GroupLayout.PREFERRED_SIZE, 132,
javax.swing.GroupLayout.PREFERRED_SIZE))
```

```
.addGap(0, 0, Short.MAX_VALUE)))
.addContainerGap())
);
layout.setVerticalGroup(
layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
.addGroup(layout.createSequentialGroup()
.addContainerGap()
.addComponent(jLabel1, javax.swing.GroupLayout.PREFERRED_SIZE, 42,
javax.swing.GroupLayout.PREFERRED_SIZE)
.addGap(18, 18, 18)
.addComponent(BtnGuest, javax.swing.GroupLayout.PREFERRED_SIZE, 23,
javax.swing.GroupLayout.PREFERRED_SIZE)
.addGap(18, 18, 18)
.addComponent(BtnReservation)
.addGap(18, 18, 18)
.addComponent(BtnRoom)
.addGap(18, 18, 18)
.addComponent(BtnPayment)
.addGap(18, 18, 18)
.addComponent(BtnExit)
.addContainerGap(141, Short.MAX_VALUE))
);

pack();
}// </editor-fold>//GEN-END:initComponents

private void BtnGuestActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
FIRST:event_BtnGuestActionPerformed
GuestFrame gf = new GuestFrame();
gf.setVisible(true);
}//GEN-LAST:event_BtnGuestActionPerformed

private void BtnReservationActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
FIRST:event_BtnReservationActionPerformed
ReservationFrame ref = new ReservationFrame();
ref.setVisible(true);
}//GEN-LAST:event_BtnReservationActionPerformed

private void BtnRoomActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
FIRST:event_BtnRoomActionPerformed
RoomFrame rf = new RoomFrame();
rf.setVisible(true);
}//GEN-LAST:event_BtnRoomActionPerformed
```

```
private void BtnPaymentActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
FIRST:event_BtnPaymentActionPerformed
PaymentFrame pf = new PaymentFrame();
pf.setVisible(true);
}//GEN-LAST:event_BtnPaymentActionPerformed

private void BtnExitActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
FIRST:event_BtnExitActionPerformed
System.exit(0);
}//GEN-LAST:event_BtnExitActionPerformed

/**
* @param args the command line arguments
*/
public static void main(String args[]) {
/* Set the Nimbus look and feel */
//<editor-fold defaultstate="collapsed" desc=" Look and feel setting code (optional) ">
/* If Nimbus (introduced in Java SE 6) is not available, stay with the default look and feel.
* For details see http://download.oracle.com/javase/tutorial/uiswing/lookandfeel/plaf.html
*/
try {
for (javax.swing.UIManager.LookAndFeelInfo info :
javax.swing.UIManager.getInstalledLookAndFeels()) {
if ("Nimbus".equals(info.getName())) {
javax.swing.UIManager.setLookAndFeel(info.getClassName());
break;
}
}
} catch (ClassNotFoundException ex) {
java.util.logging.Logger.getLogger(MainMenu.class.getName()).log(java.util.logging.Level.S
EVERE, null, ex);
} catch (InstantiationException ex) {
java.util.logging.Logger.getLogger(MainMenu.class.getName()).log(java.util.logging.Level.S
EVERE, null, ex);
} catch (IllegalAccessException ex) {
java.util.logging.Logger.getLogger(MainMenu.class.getName()).log(java.util.logging.Level.S
EVERE, null, ex);
} catch (javax.swing.UnsupportedLookAndFeelException ex) {
java.util.logging.Logger.getLogger(MainMenu.class.getName()).log(java.util.logging.Level.S
EVERE, null, ex);
}
//</editor-fold>
```

```
/* Create and display the form */

new MainMenu().setVisible(true);

}

// Variables declaration - do not modify//GEN-BEGIN:variables
private javax.swing.JButton BtnExit;
private javax.swing.JButton BtnGuest;
private javax.swing.JButton BtnPayment;
private javax.swing.JButton BtnReservation;
private javax.swing.JButton BtnRoom;
private javax.swing.JLabel jLabel1;
// End of variables declaration//GEN-END:variables
}
```

```
/*
* Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
* Click nbfs://nbhost/SystemFileSystem/Templates/GUIForms/JFrame.java to edit this template
*/
package com.mycompany.GUI;
import com.mycompany.Backend.Payment;
import com.mycompany.Backend.PaymentService;
import java.time.LocalDate;
import java.util.List;
import javax.swing.*;
import javax.swing.table.DefaultTableModel;

/**
*
* @author KGCAS
*/
public class PaymentFrame extends javax.swing.JFrame {
private PaymentService paymentService = new PaymentService();
/**
* Creates new form PaymentFrame
*/
public PaymentFrame() {
```

```
initComponents();
}

/**
* This method is called from within the constructor to initialize the form.
* WARNING: Do NOT modify this code. The content of this method is always
* regenerated by the Form Editor.
*/
@SuppressWarnings("unchecked")
// <editor-fold defaultstate="collapsed" desc="Generated Code">//GEN-
BEGIN:initComponents
private void initComponents() {

jLabel1 = new javax.swing.JLabel();
lblPaymentId = new javax.swing.JLabel();
lblReservationId = new javax.swing.JLabel();
lblAmount = new javax.swing.JLabel();
lblMethod = new javax.swing.JLabel();
lblDate = new javax.swing.JLabel();
txtPaymentId = new javax.swing.JTextField();
txtReservationId = new javax.swing.JTextField();
txtAmount = new javax.swing.JTextField();
txtDate = new javax.swing.JTextField();
cmbMethod = new javax.swing.JComboBox<>();
BtnAddPayment = new javax.swing.JButton();
BtnViewPayments = new javax.swing.JButton();
BtnDeletePayment = new javax.swing.JButton();
BtnExit = new javax.swing.JButton();
jScrollPane1 = new javax.swing.JScrollPane();
tblPayments = new javax.swing.JTable();

setDefaultCloseOperation(javax.swing.WindowConstants.EXIT_ON_CLOSE);

jLabel1.setFont(new java.awt.Font("Segoe UI", 1, 18)); // NOI18N
jLabel1.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
jLabel1.setText("Payment");

lblPaymentId.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);
lblPaymentId.setText(" Payment ID");

lblReservationId.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);
lblReservationId.setText(" Reservation Id");
```

```
lblAmount.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);
lblAmount.setText(" Amount");

lblMethod.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);
lblMethod.setText(" Method");

lblDate.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);
lblDate.setText(" Date");

cmbMethod.setModel(new javax.swing.DefaultComboBoxModel<>(new String[] { "Select
Method", "Card", "Cash", "UPI" }));

BtnAddPayment.setText("Add Payment");
BtnAddPayment.addActionListener(new java.awt.event.ActionListener() {
public void actionPerformed(java.awt.event.ActionEvent evt) {
BtnAddPaymentActionPerformed(evt);
}
});

BtnViewPayments.setText("View Payment");
BtnViewPayments.addActionListener(new java.awt.event.ActionListener() {
public void actionPerformed(java.awt.event.ActionEvent evt) {
BtnViewPaymentsActionPerformed(evt);
}
});

BtnDeletePayment.setText("Delete Payment");
BtnDeletePayment.addActionListener(new java.awt.event.ActionListener() {
public void actionPerformed(java.awt.event.ActionEvent evt) {
BtnDeletePaymentActionPerformed(evt);
}
});

BtnExit.setText("Exit");
BtnExit.addActionListener(new java.awt.event.ActionListener() {
public void actionPerformed(java.awt.event.ActionEvent evt) {
BtnExitActionPerformed(evt);
}
});

tblPayments.setModel(new javax.swing.table.DefaultTableModel(
new Object [][] {
{null, null, null, null, null},
```

```
{null, null, null, null, null},
{null, null, null, null, null},
{null, null, null, null, null}
},
new String [] {
"Payment ID", "Reservation ID", "Amount", "Method", "Date"
}
));
jScrollPane1.setViewportView(tblPayments);

javax.swing.GroupLayout layout = new javax.swing.GroupLayout(getContentPane());
getContentPane().setLayout(layout);
layout.setHorizontalGroup(
layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
.addGroup(layout.createSequentialGroup()
.addContainerGap()
.addComponent(jLabel1, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
.addGap(486, 486, 486))
.addGroup(layout.createSequentialGroup()
.addGap(62, 62, 62)
.addComponent(BtnAddPayment, javax.swing.GroupLayout.PREFERRED_SIZE, 150,
javax.swing.GroupLayout.PREFERRED_SIZE)
.addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED, 110,
Short.MAX_VALUE)
.addComponent(BtnViewPayments, javax.swing.GroupLayout.PREFERRED_SIZE, 150,
javax.swing.GroupLayout.PREFERRED_SIZE)
.addGap(101, 101, 101)
.addComponent(BtnDeletePayment, javax.swing.GroupLayout.PREFERRED_SIZE, 150,
javax.swing.GroupLayout.PREFERRED_SIZE)
.addGap(93, 93, 93)
.addComponent(BtnExit, javax.swing.GroupLayout.PREFERRED_SIZE, 150,
javax.swing.GroupLayout.PREFERRED_SIZE)
.addGap(144, 144, 144))
.addGroup(layout.createSequentialGroup()
.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
.addComponent(lblDate, javax.swing.GroupLayout.PREFERRED_SIZE, 65,
javax.swing.GroupLayout.PREFERRED_SIZE)
.addComponent(lblAmount, javax.swing.GroupLayout.PREFERRED_SIZE, 65,
javax.swing.GroupLayout.PREFERRED_SIZE)
.addGroup(layout.createSequentialGroup()
.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING,
false)
```

```
.addComponent(lblMethod, javax.swing.GroupLayout.PREFERRED_SIZE, 65,
javax.swing.GroupLayout.PREFERRED_SIZE)
.addComponent(lblReservationId, javax.swing.GroupLayout.DEFAULT_SIZE, 97,
Short.MAX_VALUE)
.addComponent(lblPaymentId, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE))
.addGap(58, 58, 58)
.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.TRAILING)
.addComponent(txtDate, javax.swing.GroupLayout.PREFERRED_SIZE, 112,
javax.swing.GroupLayout.PREFERRED_SIZE)
.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
.addComponent(txtReservationId, javax.swing.GroupLayout.PREFERRED_SIZE, 112,
javax.swing.GroupLayout.PREFERRED_SIZE)
.addComponent(txtPaymentId, javax.swing.GroupLayout.PREFERRED_SIZE, 112,
javax.swing.GroupLayout.PREFERRED_SIZE)
.addComponent(txtAmount, javax.swing.GroupLayout.PREFERRED_SIZE, 112,
javax.swing.GroupLayout.PREFERRED_SIZE))
.addComponent(cmbMethod, javax.swing.GroupLayout.Alignment.LEADING,
javax.swing.GroupLayout.PREFERRED_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.PREFERRED_SIZE))))
.addGap(76, 76, 76)
.addComponent(jScrollPane1, javax.swing.GroupLayout.PREFERRED_SIZE, 646,
javax.swing.GroupLayout.PREFERRED_SIZE)
.addGap(0, 0, Short.MAX_VALUE))
);
layout.setVerticalGroup(
layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
.addGroup(layout.createSequentialGroup()
.addContainerGap()
.addComponent(jLabel1, javax.swing.GroupLayout.PREFERRED_SIZE, 51,
javax.swing.GroupLayout.PREFERRED_SIZE)
.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
.addGroup(layout.createSequentialGroup()
.addGap(26, 26, 26)
.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
.addComponent(lblPaymentId, javax.swing.GroupLayout.PREFERRED_SIZE, 30,
javax.swing.GroupLayout.PREFERRED_SIZE)
.addComponent(txtPaymentId, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.PREFERRED_SIZE))
.addGap(18, 18, 18)
.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
.addComponent(lblReservationId, javax.swing.GroupLayout.PREFERRED_SIZE, 30,
```

```
javax.swing.GroupLayout.PREFERRED_SIZE)
.addComponent(txtReservationId, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.PREFERRED_SIZE))
.addGap(18, 18, 18)
.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
.addComponent(lblAmount, javax.swing.GroupLayout.PREFERRED_SIZE, 30,
javax.swing.GroupLayout.PREFERRED_SIZE)
.addComponent(txtAmount, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.PREFERRED_SIZE))
.addGap(18, 18, 18)
.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
.addComponent(lblMethod, javax.swing.GroupLayout.PREFERRED_SIZE, 30,
javax.swing.GroupLayout.PREFERRED_SIZE)
.addComponent(cmbMethod, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.PREFERRED_SIZE))
.addGap(18, 18, 18)
.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
.addComponent(lblDate, javax.swing.GroupLayout.PREFERRED_SIZE, 30,
javax.swing.GroupLayout.PREFERRED_SIZE)
.addComponent(txtDate, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.PREFERRED_SIZE)))
.addGroup(javax.swing.GroupLayout.Alignment.TRAILING, layout.createSequentialGroup()
.addGap(18, 18, 18)
.addComponent(jScrollPane1, javax.swing.GroupLayout.PREFERRED_SIZE, 230,
javax.swing.GroupLayout.PREFERRED_SIZE)))
.addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED, 123,
Short.MAX_VALUE)
.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
.addComponent(BtnDeletePayment, javax.swing.GroupLayout.PREFERRED_SIZE, 40,
javax.swing.GroupLayout.PREFERRED_SIZE)
.addComponent(BtnExit, javax.swing.GroupLayout.PREFERRED_SIZE, 40,
javax.swing.GroupLayout.PREFERRED_SIZE)
.addComponent(BtnAddPayment, javax.swing.GroupLayout.PREFERRED_SIZE, 40,
javax.swing.GroupLayout.PREFERRED_SIZE)
.addComponent(BtnViewPayments, javax.swing.GroupLayout.PREFERRED_SIZE, 40,
javax.swing.GroupLayout.PREFERRED_SIZE))
.addGap(37, 37, 37))
);
```

```
pack();
}// </editor-fold>//GEN-END:initComponents

private void BtnAddPaymentActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
FIRST:event_BtnAddPaymentActionPerformed
try {
String method = cmbMethod.getSelectedItem().toString();
if (method.equals("Select item")) {
JOptionPane.showMessageDialog(this, "Please select a payment method!", "Warning",
JOptionPane.WARNING_MESSAGE);
return;
}

Payment p = new Payment(
Integer.parseInt(txtPaymentId.getText()),
Integer.parseInt(txtReservationId.getText()),
Double.parseDouble(txtAmount.getText()),
method,
LocalDate.parse(txtDate.getText()) // expects yyyy-MM-dd
);

paymentService.addPayment(p);
JOptionPane.showMessageDialog(this, "Payment added successfully!", "Success",
JOptionPane.INFORMATION_MESSAGE);

// Reset fields
txtPaymentId.setText("");
txtReservationId.setText("");
txtAmount.setText("");
txtDate.setText("");
cmbMethod.setSelectedIndex(0); // back to "Select item"
} catch (Exception e) {
JOptionPane.showMessageDialog(this, "Invalid input: " + e.getMessage(), "Error",
JOptionPane.ERROR_MESSAGE);
}
}//GEN-LAST:event_BtnAddPaymentActionPerformed

private void BtnViewPaymentsActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
FIRST:event_BtnViewPaymentsActionPerformed
loadPaymentsIntoTable();
JOptionPane.showMessageDialog(this, "Payments loaded successfully!", "Info",
JOptionPane.INFORMATION_MESSAGE);
}//GEN-LAST:event_BtnViewPaymentsActionPerformed
```

```
private void BtnDeletePaymentActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
FIRST:event_BtnDeletePaymentActionPerformed
// TODO add your handling code here:
try {
// Get the ID from the text field
int id = Integer.parseInt(txtPaymentId.getText());

// Call backend service to delete
paymentService.deletePayment(id);

// Show confirmation
JOptionPane.showMessageDialog(this, "Payment deleted successfully!", "Success",
JOptionPane.INFORMATION_MESSAGE);

// Refresh table so user sees updated list
loadPaymentsIntoTable();

// Clear the input fields
txtPaymentId.setText("");
txtReservationId.setText("");
txtAmount.setText("");
txtDate.setText("");
cmbMethod.setSelectedIndex(0); // reset to "Select item"
} catch (Exception e) {
JOptionPane.showMessageDialog(this, "Invalid ID: " + e.getMessage(), "Error",
JOptionPane.ERROR_MESSAGE);
}

}//GEN-LAST:event_BtnDeletePaymentActionPerformed

private void BtnExitActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
FIRST:event_BtnExitActionPerformed
// TODO add your handling code here:
dispose();
}//GEN-LAST:event_BtnExitActionPerformed

/**
* @param args the command line arguments
*/
private void loadPaymentsIntoTable() {
DefaultTableModel model = (DefaultTableModel) tblPayments.getModel();
model.setRowCount(0); // clear table
```

```
for (Payment p : paymentService.getAllPayments()) {
model.addRow(new Object[]{
p.getId(),
p.getReservationId(),
p.getAmount(),
p.getMethod(),
p.getDate()
});
}
}

public static void main(String args[]) {
/* Set the Nimbus look and feel */
//<editor-fold defaultstate="collapsed" desc=" Look and feel setting code (optional) ">
/* If Nimbus (introduced in Java SE 6) is not available, stay with the default look and feel.
* For details see http://download.oracle.com/javase/tutorial/uiswing/lookandfeel/plaf.html
*/
try {
for (javax.swing.UIManager.LookAndFeelInfo info :
javax.swing.UIManager.getInstalledLookAndFeels()) {
if ("Nimbus".equals(info.getName())) {
javax.swing.UIManager.setLookAndFeel(info.getClassName());
break;
}
}
} catch (ClassNotFoundException ex) {
java.util.logging.Logger.getLogger(PaymentFrame.class.getName()).log(java.util.logging.Le
vel.SEVERE, null, ex);
} catch (InstantiationException ex) {
java.util.logging.Logger.getLogger(PaymentFrame.class.getName()).log(java.util.logging.Le
vel.SEVERE, null, ex);
} catch (IllegalAccessException ex) {
java.util.logging.Logger.getLogger(PaymentFrame.class.getName()).log(java.util.logging.Le
vel.SEVERE, null, ex);
} catch (javax.swing.UnsupportedLookAndFeelException ex) {
java.util.logging.Logger.getLogger(PaymentFrame.class.getName()).log(java.util.logging.Le
vel.SEVERE, null, ex);
}
//</editor-fold>

/* Create and display the form */
java.awt.EventQueue.invokeLater(new Runnable() {
```

```
public void run() {
new PaymentFrame().setVisible(true);
}
});
}

// Variables declaration - do not modify//GEN-BEGIN:variables
private javax.swing.JButton BtnAddPayment;
private javax.swing.JButton BtnDeletePayment;
private javax.swing.JButton BtnExit;
private javax.swing.JButton BtnViewPayments;
private javax.swing.JComboBox<String> cmbMethod;
private javax.swing.JLabel jLabel1;
private javax.swing.JScrollPane jScrollPane1;
private javax.swing.JLabel lblAmount;
private javax.swing.JLabel lblDate;
private javax.swing.JLabel lblMethod;
private javax.swing.JLabel lblPaymentId;
private javax.swing.JLabel lblReservationId;
private javax.swing.JTable tblPayments;
private javax.swing.JTextField txtAmount;
private javax.swing.JTextField txtDate;
private javax.swing.JTextField txtPaymentId;
private javax.swing.JTextField txtReservationId;
// End of variables declaration//GEN-END:variables
}
```

```
/*
* Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
* Click nbfs://nbhost/SystemFileSystem/Templates/GUIForms/JFrame.java to edit this template
*/
package com.mycompany.GUI;
import com.mycompany.Backend.ReservationService;
import com.mycompany.Backend.Reservation;
import javax.swing.JOptionPane;
import javax.swing.table.DefaultTableModel;
import java.time.LocalDate;
```

```
/**
*
* @author KGCAS
*/
public class ReservationFrame extends javax.swing.JFrame {
private final ReservationService reservationService = new ReservationService();
/**
* Creates new form ReservationFrame
*/
public ReservationFrame() {
initComponents();
}

/**
* This method is called from within the constructor to initialize the form.
* WARNING: Do NOT modify this code. The content of this method is always
* regenerated by the Form Editor.
*/
@SuppressWarnings("unchecked")
// <editor-fold defaultstate="collapsed" desc="Generated Code">//GEN-
BEGIN:initComponents
private void initComponents() {

jScrollPane1 = new javax.swing.JScrollPane();
jTable1 = new javax.swing.JTable();
jLabel1 = new javax.swing.JLabel();
lblReservationId = new javax.swing.JLabel();
lblGuestId = new javax.swing.JLabel();
lblRoomId = new javax.swing.JLabel();
lblCheckInDate = new javax.swing.JLabel();
lblCheckOutDate = new javax.swing.JLabel();
txtReservationId = new javax.swing.JTextField();
txtGuestId = new javax.swing.JTextField();
txtRoomNumber = new javax.swing.JTextField();
txtCheckInDate = new javax.swing.JTextField();
txtCheckOutDate = new javax.swing.JTextField();
BtnAddReservation = new javax.swing.JButton();
BtnDeleteReservation = new javax.swing.JButton();
BtnViewReservation = new javax.swing.JButton();
BtnExit = new javax.swing.JButton();
jScrollPane2 = new javax.swing.JScrollPane();
tblReservations = new javax.swing.JTable();
```

```
jTable1.setModel(new javax.swing.table.DefaultTableModel(
new Object [][] {
{null, null, null, null},
{null, null, null, null},
{null, null, null, null},
{null, null, null, null}
},
new String [] {
"Title 1", "Title 2", "Title 3", "Title 4"
}
));
jScrollPane1.setViewportView(jTable1);

setDefaultCloseOperation(javax.swing.WindowConstants.EXIT_ON_CLOSE);

jLabel1.setFont(new java.awt.Font("Segoe UI", 1, 18)); // NOI18N
jLabel1.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
jLabel1.setText("Reservation");

lblReservationId.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);
lblReservationId.setText("Reservation ID");

lblGuestId.setText("Guest ID");

lblRoomId.setText("Room ID");

lblCheckInDate.setText("Check in Date");

lblCheckOutDate.setText("Check out Date");

BtnAddReservation.setText("Add Reservation");
BtnAddReservation.addActionListener(new java.awt.event.ActionListener() {
public void actionPerformed(java.awt.event.ActionEvent evt) {
BtnAddReservationActionPerformed(evt);
}
});

BtnDeleteReservation.setText("Delete Reservation");
BtnDeleteReservation.addActionListener(new java.awt.event.ActionListener() {
public void actionPerformed(java.awt.event.ActionEvent evt) {
BtnDeleteReservationActionPerformed(evt);
}
});
```

```
BtnViewReservation.setText("View Reservations ");
BtnViewReservation.addActionListener(new java.awt.event.ActionListener() {
public void actionPerformed(java.awt.event.ActionEvent evt) {
BtnViewReservationActionPerformed(evt);
}
});

BtnExit.setText("Exit");
BtnExit.addActionListener(new java.awt.event.ActionListener() {
public void actionPerformed(java.awt.event.ActionEvent evt) {
BtnExitActionPerformed(evt);
}
});

tblReservations.setModel(new javax.swing.table.DefaultTableModel(
new Object [][] {
{null, null, null, null, null},
{null, null, null, null, null},
{null, null, null, null, null},
{null, null, null, null, null}
},
new String [] {
"Reservation ID", "Guest ID", "Room ID", "Check in", "Check out"
}
));
jScrollPane2.setViewportView(tblReservations);

javax.swing.GroupLayout layout = new javax.swing.GroupLayout(getContentPane());
getContentPane().setLayout(layout);
layout.setHorizontalGroup(
layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
.addGroup(layout.createSequentialGroup()
.addContainerGap()
.addComponent(jLabel1, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
.addGap(565, 565, 565))
.addGroup(javax.swing.GroupLayout.Alignment.TRAILING, layout.createSequentialGroup()
.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.TRAILING)
.addGroup(layout.createSequentialGroup()
.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
.addComponent(lblReservationId, javax.swing.GroupLayout.PREFERRED_SIZE, 85,
javax.swing.GroupLayout.PREFERRED_SIZE)
```

```
.addComponent(lblGuestId, javax.swing.GroupLayout.PREFERRED_SIZE, 85,
javax.swing.GroupLayout.PREFERRED_SIZE))
.addGap(64, 64, 64)
.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING,
false)
.addComponent(txtReservationId, javax.swing.GroupLayout.DEFAULT_SIZE, 136,
Short.MAX_VALUE)
.addComponent(txtGuestId)
.addComponent(txtRoomNumber)
.addComponent(txtCheckInDate)
.addComponent(txtCheckOutDate))
.addGap(18, 18, 18)
.addComponent(jScrollPane2))
.addGroup(layout.createSequentialGroup()
.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
.addComponent(lblRoomId, javax.swing.GroupLayout.PREFERRED_SIZE, 85,
javax.swing.GroupLayout.PREFERRED_SIZE)
.addComponent(lblCheckInDate, javax.swing.GroupLayout.PREFERRED_SIZE, 85,
javax.swing.GroupLayout.PREFERRED_SIZE)
.addComponent(lblCheckOutDate, javax.swing.GroupLayout.PREFERRED_SIZE, 85,
javax.swing.GroupLayout.PREFERRED_SIZE)
.addComponent(BtnAddReservation, javax.swing.GroupLayout.PREFERRED_SIZE, 150,
javax.swing.GroupLayout.PREFERRED_SIZE))
.addGap(81, 81, 81)
.addComponent(BtnDeleteReservation, javax.swing.GroupLayout.PREFERRED_SIZE, 150,
javax.swing.GroupLayout.PREFERRED_SIZE)
.addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED, 95,
Short.MAX_VALUE)
.addComponent(BtnViewReservation, javax.swing.GroupLayout.PREFERRED_SIZE, 150,
javax.swing.GroupLayout.PREFERRED_SIZE)
.addGap(79, 79, 79)
.addComponent(BtnExit, javax.swing.GroupLayout.PREFERRED_SIZE, 150,
javax.swing.GroupLayout.PREFERRED_SIZE)))
.addGap(25, 25, 25))
);
layout.setVerticalGroup(
layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
.addGroup(layout.createSequentialGroup()
.addContainerGap()
.addComponent(jLabel1, javax.swing.GroupLayout.PREFERRED_SIZE, 35,
javax.swing.GroupLayout.PREFERRED_SIZE)
.addGap(12, 12, 12)
.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
```

```
.addGroup(layout.createSequentialGroup()
.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
.addComponent(lblReservationId)
.addComponent(txtReservationId, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.PREFERRED_SIZE))
.addGap(18, 18, 18)
.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
.addComponent(lblGuestId)
.addComponent(txtGuestId, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.PREFERRED_SIZE))
.addGap(18, 18, 18)
.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
.addComponent(lblRoomId)
.addComponent(txtRoomNumber, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.PREFERRED_SIZE))
.addGap(18, 18, 18)
.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
.addComponent(lblCheckInDate)
.addComponent(txtCheckInDate, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.PREFERRED_SIZE))
.addGap(18, 18, 18)
.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
.addComponent(lblCheckOutDate)
.addComponent(txtCheckOutDate, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.PREFERRED_SIZE)))
.addComponent(jScrollPane2, javax.swing.GroupLayout.Alignment.TRAILING,
javax.swing.GroupLayout.PREFERRED_SIZE, 182,
javax.swing.GroupLayout.PREFERRED_SIZE))
.addGap(18, 18, Short.MAX_VALUE)
.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
.addComponent(BtnAddReservation, javax.swing.GroupLayout.PREFERRED_SIZE, 40,
javax.swing.GroupLayout.PREFERRED_SIZE)
.addComponent(BtnDeleteReservation, javax.swing.GroupLayout.PREFERRED_SIZE, 40,
javax.swing.GroupLayout.PREFERRED_SIZE)
.addComponent(BtnViewReservation, javax.swing.GroupLayout.PREFERRED_SIZE, 40,
javax.swing.GroupLayout.PREFERRED_SIZE)
.addComponent(BtnExit, javax.swing.GroupLayout.PREFERRED_SIZE, 40,
javax.swing.GroupLayout.PREFERRED_SIZE))
```

```
.addGap(13, 13, 13))
);

pack();
}// </editor-fold>//GEN-END:initComponents

private void BtnAddReservationActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
FIRST:event_BtnAddReservationActionPerformed
// TODO add your handling code here:
try {
Reservation r = new Reservation(
txtReservationId.getText(),
txtGuestId.getText(),
txtRoomNumber.getText(), // <-- updated
LocalDate.parse(txtCheckInDate.getText()),
LocalDate.parse(txtCheckOutDate.getText())
);

reservationService.addReservation(r);
JOptionPane.showMessageDialog(this, "Reservation added successfully!");
loadReservationsIntoTable();
} catch (Exception e) {
JOptionPane.showMessageDialog(this, "Error: " + e.getMessage());
}
}//GEN-LAST:event_BtnAddReservationActionPerformed

private void BtnDeleteReservationActionPerformed(java.awt.event.ActionEvent evt)
{//GEN-FIRST:event_BtnDeleteReservationActionPerformed
// TODO add your handling code here:
try {
String reservationId = txtReservationId.getText();
reservationService.deleteReservation(reservationId);
JOptionPane.showMessageDialog(this, "Reservation deleted successfully!", "Success",
JOptionPane.INFORMATION_MESSAGE);

loadReservationsIntoTable(); // refresh table
} catch (Exception e) {
JOptionPane.showMessageDialog(this, "Invalid ID: " + e.getMessage(), "Error",
JOptionPane.ERROR_MESSAGE);
}
}//GEN-LAST:event_BtnDeleteReservationActionPerformed

private void BtnViewReservationActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
```

```
FIRST:event_BtnViewReservationActionPerformed
// TODO add your handling code here:
loadReservationsIntoTable();
}//GEN-LAST:event_BtnViewReservationActionPerformed

private void BtnExitActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
FIRST:event_BtnExitActionPerformed
// TODO add your handling code here:
dispose();
}//GEN-LAST:event_BtnExitActionPerformed

/**
* @param args the command line arguments
*/
private void loadReservationsIntoTable() {
DefaultTableModel model = (DefaultTableModel) tblReservations.getModel();
model.setRowCount(0); // clear table

for (Reservation r : reservationService.getAllReservations()) {
model.addRow(new Object[]{
r.getReservationId(),
r.getGuestId(),
r.getRoomNumber(),
r.getCheckIn(),
r.getCheckOut()
});
}
}
public static void main(String args[]) {
/* Set the Nimbus look and feel */
//<editor-fold defaultstate="collapsed" desc=" Look and feel setting code (optional) ">
/* If Nimbus (introduced in Java SE 6) is not available, stay with the default look and feel.
* For details see http://download.oracle.com/javase/tutorial/uiswing/lookandfeel/plaf.html
*/
try {
for (javax.swing.UIManager.LookAndFeelInfo info :
javax.swing.UIManager.getInstalledLookAndFeels()) {
if ("Nimbus".equals(info.getName())) {
javax.swing.UIManager.setLookAndFeel(info.getClassName());
break;
}
}
} catch (ClassNotFoundException ex) {
```

```
java.util.logging.Logger.getLogger(ReservationFrame.class.getName()).log(java.util.logging.
Level.SEVERE, null, ex);
} catch (InstantiationException ex) {
java.util.logging.Logger.getLogger(ReservationFrame.class.getName()).log(java.util.logging.
Level.SEVERE, null, ex);
} catch (IllegalAccessException ex) {
java.util.logging.Logger.getLogger(ReservationFrame.class.getName()).log(java.util.logging.
Level.SEVERE, null, ex);
} catch (javax.swing.UnsupportedLookAndFeelException ex) {
java.util.logging.Logger.getLogger(ReservationFrame.class.getName()).log(java.util.logging.
Level.SEVERE, null, ex);
}
//</editor-fold>

/* Create and display the form */
java.awt.EventQueue.invokeLater(new Runnable() {
public void run() {
new ReservationFrame().setVisible(true);
}
});
}

// Variables declaration - do not modify//GEN-BEGIN:variables
private javax.swing.JButton BtnAddReservation;
private javax.swing.JButton BtnDeleteReservation;
private javax.swing.JButton BtnExit;
private javax.swing.JButton BtnViewReservation;
private javax.swing.JLabel jLabel1;
private javax.swing.JScrollPane jScrollPane1;
private javax.swing.JScrollPane jScrollPane2;
private javax.swing.JTable jTable1;
private javax.swing.JLabel lblCheckInDate;
private javax.swing.JLabel lblCheckOutDate;
private javax.swing.JLabel lblGuestId;
private javax.swing.JLabel lblReservationId;
private javax.swing.JLabel lblRoomId;
private javax.swing.JTable tblReservations;
private javax.swing.JTextField txtCheckInDate;
private javax.swing.JTextField txtCheckOutDate;
private javax.swing.JTextField txtGuestId;
private javax.swing.JTextField txtReservationId;
private javax.swing.JTextField txtRoomNumber;
```

```
// End of variables declaration//GEN-END:variables
}
```

```
/*
* Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this
license
* Click nbfs://nbhost/SystemFileSystem/Templates/GUIForms/JFrame.java to edit this
template
*/
package com.mycompany.GUI;

import com.mycompany.Backend.RoomService;
import com.mycompany.Backend.Room;
import java.time.LocalDate;
import java.util.List;
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
/**
*
* @author KGCAS
*/
public class RoomFrame extends javax.swing.JFrame {
private RoomService roomService = new RoomService();
/**
* Creates new form RoomFrame
*/
public RoomFrame() {
initComponents();
}

/**
* This method is called from within the constructor to initialize the form.
* WARNING: Do NOT modify this code. The content of this method is always
* regenerated by the Form Editor.
*/
@SuppressWarnings("unchecked")
// <editor-fold defaultstate="collapsed" desc="Generated Code">//GEN-
BEGIN:initComponents
private void initComponents() {
```

```
jLabel1 = new javax.swing.JLabel();
jLabel2 = new javax.swing.JLabel();
jLabel3 = new javax.swing.JLabel();
jLabel4 = new javax.swing.JLabel();
jLabel5 = new javax.swing.JLabel();
txtRoomNumber = new javax.swing.JTextField();
txtRoomType = new javax.swing.JTextField();
txtRoomPrice = new javax.swing.JTextField();
cmdStatus = new javax.swing.JComboBox<>();
jScrollPane1 = new javax.swing.JScrollPane();
TblRoom = new javax.swing.JTable();
BtnAddRoom = new javax.swing.JButton();
BtnViewRoom = new javax.swing.JButton();
BtnDeleteRoom = new javax.swing.JButton();
BtnExit = new javax.swing.JButton();

setDefaultCloseOperation(javax.swing.WindowConstants.EXIT_ON_CLOSE);

jLabel1.setFont(new java.awt.Font("Segoe UI", 1, 18)); // NOI18N
jLabel1.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
jLabel1.setText("Rooms");

jLabel2.setText("Room Number");
jLabel2.setToolTipText("");

jLabel3.setText("Room Type");

jLabel4.setText("Room Price");

jLabel5.setText("Status");

txtRoomPrice.addActionListener(new java.awt.event.ActionListener() {
public void actionPerformed(java.awt.event.ActionEvent evt) {
txtRoomPriceActionPerformed(evt);
}
});

cmdStatus.setModel(new javax.swing.DefaultComboBoxModel<>(new String[] { "Select",
"Available", "Unavailable" }));

TblRoom.setModel(new javax.swing.table.DefaultTableModel(
new Object [][] {
{null, null, null, null},
```

```
{null, null, null, null},
{null, null, null, null},
{null, null, null, null}
},
new String [] {
"Room ID", "RoomType", "Room Price", "Status"
}
));
jScrollPane1.setViewportView(TblRoom);

BtnAddRoom.setText("Add Room");
BtnAddRoom.addActionListener(new java.awt.event.ActionListener() {
public void actionPerformed(java.awt.event.ActionEvent evt) {
BtnAddRoomActionPerformed(evt);
}
});

BtnViewRoom.setText("View Rooms");
BtnViewRoom.addActionListener(new java.awt.event.ActionListener() {
public void actionPerformed(java.awt.event.ActionEvent evt) {
BtnViewRoomActionPerformed(evt);
}
});

BtnDeleteRoom.setText("Delete Room");
BtnDeleteRoom.addActionListener(new java.awt.event.ActionListener() {
public void actionPerformed(java.awt.event.ActionEvent evt) {
BtnDeleteRoomActionPerformed(evt);
}
});

BtnExit.setText("Exit");
BtnExit.addActionListener(new java.awt.event.ActionListener() {
public void actionPerformed(java.awt.event.ActionEvent evt) {
BtnExitActionPerformed(evt);
}
});

javax.swing.GroupLayout layout = new javax.swing.GroupLayout(getContentPane());
getContentPane().setLayout(layout);
layout.setHorizontalGroup(
layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
.addGroup(layout.createSequentialGroup()
```

```
.addContainerGap()
.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
.addComponent(jLabel1, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
.addGroup(layout.createSequentialGroup()
.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.TRAILING)
.addGroup(layout.createSequentialGroup()
.addComponent(BtnAddRoom)
.addGap(44, 44, 44)
.addComponent(BtnViewRoom))
.addGroup(layout.createSequentialGroup()
.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING,
false)
.addComponent(jLabel3, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
.addComponent(jLabel2, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
.addComponent(jLabel4, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
.addComponent(jLabel5, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE))
.addGap(36, 36, 36)
.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING,
false)
.addComponent(cmdStatus, 0, javax.swing.GroupLayout.DEFAULT_SIZE,
Short.MAX_VALUE)
.addComponent(txtRoomPrice)
.addComponent(txtRoomType)
.addComponent(txtRoomNumber))))
.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
.addGroup(layout.createSequentialGroup()
.addGap(18, 18, 18)
.addComponent(jScrollPane1, javax.swing.GroupLayout.PREFERRED_SIZE, 432,
javax.swing.GroupLayout.PREFERRED_SIZE))
.addGroup(layout.createSequentialGroup()
.addGap(46, 46, 46)
.addComponent(BtnDeleteRoom)
.addGap(41, 41, 41)
.addComponent(BtnExit)))
.addGap(0, 0, Short.MAX_VALUE)))
.addContainerGap())
);
layout.setVerticalGroup(
```

```
layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
.addGroup(layout.createSequentialGroup()
.addContainerGap()
.addComponent(jLabel1, javax.swing.GroupLayout.PREFERRED_SIZE, 42,
javax.swing.GroupLayout.PREFERRED_SIZE)
.addGap(18, 18, 18)
.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING,
false)
.addGroup(layout.createSequentialGroup()
.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
.addComponent(jLabel2)
.addComponent(txtRoomNumber, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.PREFERRED_SIZE))
.addGap(18, 18, 18)
.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
.addComponent(jLabel3)
.addComponent(txtRoomType, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.PREFERRED_SIZE))
.addGap(18, 18, 18)
.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
.addComponent(jLabel4)
.addComponent(txtRoomPrice, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.PREFERRED_SIZE))
.addGap(18, 18, 18)
.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
.addComponent(jLabel5)
.addComponent(cmdStatus, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.PREFERRED_SIZE)))
.addComponent(jScrollPane1, javax.swing.GroupLayout.PREFERRED_SIZE, 0,
Short.MAX_VALUE))
.addGap(18, 18, 18)
.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
.addComponent(BtnAddRoom)
.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
.addComponent(BtnViewRoom)
.addComponent(BtnDeleteRoom)
.addComponent(BtnExit)))
.addContainerGap(13, Short.MAX_VALUE))
);
```

```
pack();
}// </editor-fold>//GEN-END:initComponents

private void txtRoomPriceActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
FIRST:event_txtRoomPriceActionPerformed
// TODO add your handling code here:

}//GEN-LAST:event_txtRoomPriceActionPerformed

private void BtnAddRoomActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
FIRST:event_BtnAddRoomActionPerformed
// TODO add your handling code here:
try {
Room r = new Room(
txtRoomNumber.getText(),
txtRoomType.getText(),
Double.parseDouble(txtRoomPrice.getText()),
cmdStatus.getSelectedItem().toString() // <-- dropdown value
);

roomService.addRoom(r);
JOptionPane.showMessageDialog(this, "Room added successfully!");
loadRoomsIntoTable();
} catch (Exception e) {
JOptionPane.showMessageDialog(this, "Error: " + e.getMessage());
}
}//GEN-LAST:event_BtnAddRoomActionPerformed

private void BtnViewRoomActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
FIRST:event_BtnViewRoomActionPerformed
// TODO add your handling code here:
loadRoomsIntoTable();
}//GEN-LAST:event_BtnViewRoomActionPerformed

private void BtnDeleteRoomActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
FIRST:event_BtnDeleteRoomActionPerformed
// TODO add your handling code here:
try {
String roomNumber = txtRoomNumber.getText();
roomService.deleteRoom(roomNumber);
JOptionPane.showMessageDialog(this, "Room deleted successfully!");
loadRoomsIntoTable();
```

```
} catch (Exception e) {
JOptionPane.showMessageDialog(this, "Error: " + e.getMessage());
}
}//GEN-LAST:event_BtnDeleteRoomActionPerformed

private void BtnExitActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
FIRST:event_BtnExitActionPerformed
// TODO add your handling code here:
dispose();
}//GEN-LAST:event_BtnExitActionPerformed

/**
* @param args the command line arguments
*/

private void loadRoomsIntoTable() {
DefaultTableModel model = (DefaultTableModel) TblRoom.getModel();
model.setRowCount(0);

for (Room r : roomService.getAllRooms()) {
model.addRow(new Object[]{
r.getNumber(),
r.getType(),
r.getPrice(),
r.getStatus()
});
}
}
public static void main(String args[]) {
/* Set the Nimbus look and feel */
//<editor-fold defaultstate="collapsed" desc=" Look and feel setting code (optional) ">
/* If Nimbus (introduced in Java SE 6) is not available, stay with the default look and feel.
* For details see http://download.oracle.com/javase/tutorial/uiswing/lookandfeel/plaf.html
*/
try {
for (javax.swing.UIManager.LookAndFeelInfo info :
javax.swing.UIManager.getInstalledLookAndFeels()) {
if ("Nimbus".equals(info.getName())) {
javax.swing.UIManager.setLookAndFeel(info.getClassName());
break;
}
}
} catch (ClassNotFoundException ex) {
```

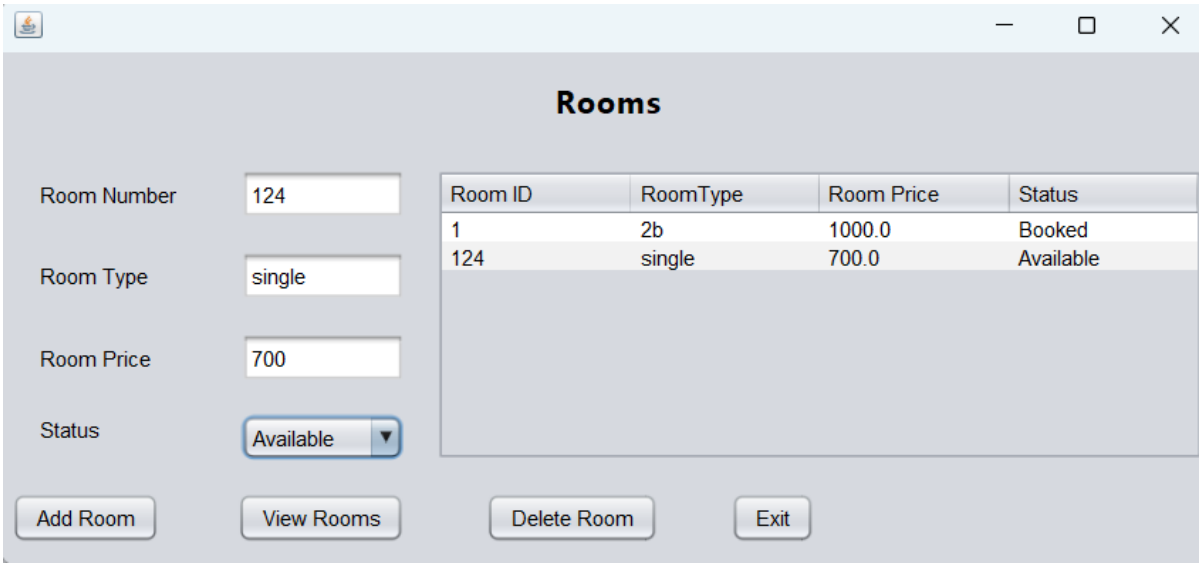
```
java.util.logging.Logger.getLogger(RoomFrame.class.getName()).log(java.util.logging.Level.
SEVERE, null, ex);
} catch (InstantiationException ex) {
java.util.logging.Logger.getLogger(RoomFrame.class.getName()).log(java.util.logging.Level.
SEVERE, null, ex);
} catch (IllegalAccessException ex) {
java.util.logging.Logger.getLogger(RoomFrame.class.getName()).log(java.util.logging.Level.
SEVERE, null, ex);
} catch (javax.swing.UnsupportedLookAndFeelException ex) {
java.util.logging.Logger.getLogger(RoomFrame.class.getName()).log(java.util.logging.Level.
SEVERE, null, ex);
}
//</editor-fold>

/* Create and display the form */
java.awt.EventQueue.invokeLater(new Runnable() {
public void run() {
new RoomFrame().setVisible(true);
}
});
}

// Variables declaration - do not modify//GEN-BEGIN:variables
private javax.swing.JButton BtnAddRoom;
private javax.swing.JButton BtnDeleteRoom;
private javax.swing.JButton BtnExit;
private javax.swing.JButton BtnViewRoom;
private javax.swing.JTable TblRoom;
private javax.swing.JComboBox<String> cmdStatus;
private javax.swing.JLabel jLabel1;
private javax.swing.JLabel jLabel2;
private javax.swing.JLabel jLabel3;
private javax.swing.JLabel jLabel4;
private javax.swing.JLabel jLabel5;
private javax.swing.JScrollPane jScrollPane1;
private javax.swing.JTextField txtRoomNumber;
private javax.swing.JTextField txtRoomPrice;
private javax.swing.JTextField txtRoomType;
// End of variables declaration//GEN-END:variables
}
```

---

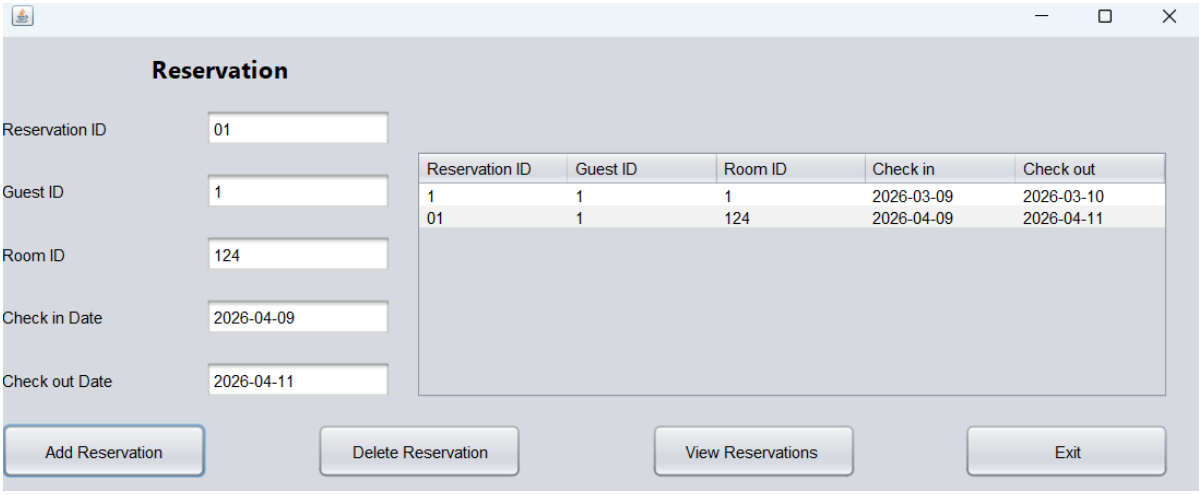
6. INPUT AND OUTPUT SCREENS

Main Menu :

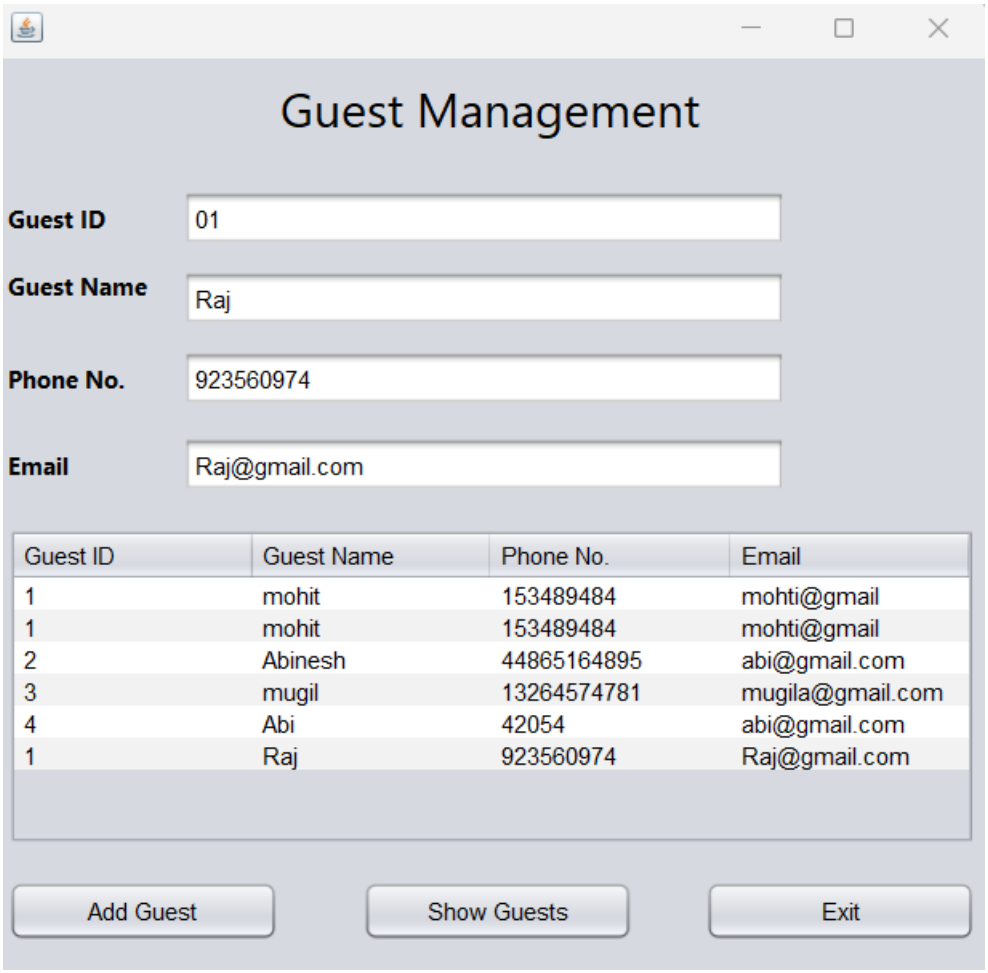
Adding Rooms :



Reservation for a guest :



Managing Guest Information :



7. DISCUSSION & CONCLUSION

7.1 Discussion

The Hotel Reservation System was developed to automate the process of managing reservations, guest records, and room availability in hotels. During the development, several Core Java concepts were applied, including classes, objects, methods, loops, conditional statements, and exception handling. The use of Java Swing provided a graphical user interface that made the system more intuitive and user-friendly compared to console-based applications.

The modular design approach ensured that each functionality was handled by a dedicated component: room availability, guest management, reservation booking, invoice generation, and exit. This separation of concerns made the system easier to test, maintain, and extend. For example, the reservation module directly interacts with the room availability module to prevent double-bookings, while the invoice module ensures accurate billing and confirmation.

Testing confirmed that the system performs reliably under different scenarios. Valid inputs produced accurate results, while invalid inputs were handled gracefully through error messages. The system successfully reduced manual effort, minimized errors, and improved efficiency in reservation handling.

From an educational perspective, the project demonstrates how object-oriented programming principles can be applied to solve real-world problems. It bridges the gap between theoretical Java concepts and practical business applications, making it a valuable learning experience.

7.2 Conclusion

The Hotel Reservation System successfully achieves its objectives by automating hotel reservation processes, reducing manual errors, and improving customer service efficiency. It provides a structured, user-friendly interface for staff to manage bookings and generate confirmations.

This project highlights the importance of program analysis, modular system design, and systematic testing in software development. It shows how Java programming can be applied to create practical solutions for the hospitality industry.

While the current version focuses on core functionalities, the system can be enhanced further by integrating a database for persistent storage, adding online booking capabilities, and incorporating payment gateways. These improvements would make the system suitable for larger hotels and multi-branch operations.

In conclusion, the project not only solves immediate operational challenges but also lays a strong foundation for future development. It demonstrates the potential of Java as a tool for building scalable, efficient, and real-world business applications.

8. REFERENCES

The following resources were used during the development of this project:

Books

• *Joshua Bloch – Effective Java (3rd edition)*

Websites

• https://www.oracle.com/java
• https://www.geeksforgeeks.org
• https://www.w3schools.com